

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	A Lokanath Reddy	Reg. No.:	192425321
Section / Batch:	AIML	Date:	

Code of Conduct

I A Lokanath Reddy/192425321 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. AI-Powered Insurance Claim Processing System

An insurance company is developing an NLP-based system to automatically process customer insurance claims. The system uses **Context-Free Grammar (CFG)** to parse customer statements and extract information such as the claimant, damaged object, cause of damage, and claim amount.

However, the system faces the following problems:

- Ambiguous sentence structures
- Difficulty distinguishing noun and prepositional phrase attachment
- Subject–verb agreement errors
- Inefficient parsing of lengthy customer descriptions
- Difficulty handling conversational and incomplete input

Example customer statement:

"I reported the damage to the car after the accident."

The sentence can produce different interpretations depending on whether "**to the car**" is associated with *reported* or *damage*.

Tasks:

- Analyze the structural ambiguity in the given sentence using CFG and illustrate the possible interpretations.
- Evaluate the limitations of a basic CFG in handling ambiguity, agreement, and long customer-generated insurance statements.
- Design an improved parsing approach using **PCFG, Feature Structures, and Earley Parsing** to resolve ambiguity and efficiently process the claim.
- Justify how the proposed architecture can improve the accuracy, scalability, and reliability of automated insurance claim processing.

Answer:

- The sentence "I reported the damage to the car after the accident" is ambiguous because the PP "to the car" can attach either to the verb "reported" (as if I reported something to the car) or to the noun "damage" (damage to the car). Using CFG, both trees are grammatically valid since CFG rules like $VP \rightarrow V NP PP$ and $NP \rightarrow NP PP$ can both apply, so the grammar alone can't tell which one is meant.
- A basic CFG just checks whether a sentence is grammatically valid, it does not know which meaning actually makes sense. So it can't resolve PP attachment on its own, it can't catch subject-verb agreement mistakes properly, and for long customer sentences it becomes very slow because it keeps generating many possible trees for the same sentence.
- PCFG assigns probabilities to each rule so the parser can pick the more likely tree (here, damage-to-the-car is more probable than reporting-to-a-car). Feature Structures add extra checks like number/person agreement so

wrong structures are rejected early. Earley Parsing handles this efficiently even for long, ambiguous sentences without needing to backtrack like a simple top-down parser.

d) Combining PCFG + Feature Structures + Earley Parsing means the system picks the statistically correct interpretation, respects grammar agreement rules, and parses long claim descriptions in reasonable time. This directly improves accuracy (correct meaning extracted), scalability (handles long real customer text), and reliability (consistent extraction of claimant, object, cause, and amount) for the claim processing system.

I wrote a small program below to represent both attachment interpretations and pick the semantically valid one.

Code:

```
sentence = "I reported the damage to the car after the accident"

interpretation_1 = {
    "attachment": "to the car -> attached to REPORTED",
    "meaning": "I reported to the car, the damage after the accident",
    "parse": ("S", ("VP", "reported", ("PP", "to the car"), ("NP", "the damage"))),
    "valid": False,
}

interpretation_2 = {
    "attachment": "to the car -> attached to DAMAGE",
    "meaning": "I reported the damage(to the car) after the accident",
    "parse": ("S", ("VP", "reported", ("NP", "the damage", ("PP", "to the car")))),
    "valid": True,
}

interpretations = [interpretation_1, interpretation_2]

print("Sentence:", sentence)
print()
for i, interp in enumerate(interpretations, start=1):
    print("Interpretation", i, ":", interp["attachment"])
    print("  Meaning:", interp["meaning"])
    print("  Parse:", interp["parse"])
    print()

correct = [i for i in interpretations if i["valid"]][0]
print("Semantically Correct Interpretation:", correct["attachment"])
print("Reason: a car cannot receive a report, so PP must attach to the noun 'damage', not the verb 'reported'")
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\studies\nlp\co4ass2> python q1_insurance.py
Sentence: I reported the damage to the car after the accident

Interpretation 1 : to the car -> attached to REPORTED
Meaning: I reported to the car, the damage after the accident
Parse: ('S', ('VP', 'reported', ('PP', 'to the car'), ('NP', 'the damage'))))

Interpretation 2 : to the car -> attached to DAMAGE
Meaning: I reported the damage(to the car) after the accident
Parse: ('S', ('VP', 'reported', ('NP', 'the damage', ('PP', 'to the car')))))

Semantically Correct Interpretation: to the car -> attached to DAMAGE
Reason: a car cannot receive a report, so PP must attach to the noun 'damage', not the verb 'reported'

PS D:\studies\nlp\co4ass2>
```

2. Intelligent Railway Ticket Booking Assistant

A railway company is developing a conversational AI assistant for ticket booking. The system currently uses **top-down parsing** to interpret passenger requests.

The system encounters:

- Excessive backtracking
- Ambiguous attachment of phrases
- Incomplete passenger requests
- Long-distance dependencies
- Slow response for complex queries

Example passenger request:

"Book two tickets from Chennai to Delhi for my parents with AC sleeper."

The system sometimes incorrectly associates "**with AC sleeper**" with the passengers instead of the ticket/class information.

Tasks:

- a) Analyze the possible syntactic interpretations of the given passenger request using CFG.
- b) Evaluate why top-down parsing may result in excessive backtracking and inefficient processing for conversational ticket-booking queries.
- c) Analyze how **Earley Parsing** can handle ambiguity, incomplete input, and different possible parse structures more effectively.
- d) Compare **Top-Down Parsing, CFG-based parsing, and Earley Parsing** in terms of ambiguity handling, computational efficiency, partial-input processing, and suitability for real-time railway booking systems.

Answer:

- a) The sentence "Book two tickets from Chennai to Delhi for my parents with AC sleeper" has an attachment ambiguity on "with AC sleeper". Using CFG it can attach to "my parents" (as if parents have AC sleeper as a feature) or to "tickets" (which is the correct meaning, AC sleeper is the ticket class).

b) Top-down parsing starts from the start symbol and tries to expand rules to match the input, and if a rule choice doesn't match it has to backtrack and try another rule. For a long conversational sentence with many possible attachments (from/to, for, with), the parser keeps trying and undoing many rule choices, which makes it slow and inefficient, especially for real-time booking.

c) Earley Parsing builds a chart of possible parse states as it reads the sentence left to right, and keeps all valid partial parses together instead of committing to one and backtracking. This lets it handle ambiguity (multiple attachments) and incomplete input (like a passenger stopping mid-sentence) much more smoothly, and it also works for any type of CFG grammar without needing to rewrite the grammar.

d) Top-down parsing is simple but slow due to backtracking and struggles with ambiguity. Basic CFG parsing (like CYK) is more structured but still needs the grammar in a specific form and doesn't rank interpretations. Earley Parsing handles ambiguity, partial/incomplete input, and different sentence structures efficiently in $O(n^3)$ worst case, and is much more suitable for a real-time conversational railway booking system where users can pause or give partial info.

Below is a small demonstration comparing both interpretations and showing backtracking steps vs Earley states processed.

Code:

```
sentence = "Book two tickets from Chennai to Delhi for my parents with AC sleeper"

interpretation_1 = {
    "attachment": "with AC sleeper -> attached to PARENTS",
    "meaning": "my parents(with AC sleeper) i.e. parents described as having AC sleeper",
    "valid": False,
}

interpretation_2 = {
    "attachment": "with AC sleeper -> attached to TICKETS",
    "meaning": "tickets(with AC sleeper) i.e. ticket class is AC sleeper",
    "valid": True,
}

print("Sentence:", sentence)
print()
for i, interp in enumerate([interpretation_1, interpretation_2], start=1):
    print("Interpretation", i, ":", interp["attachment"])
    print("  Meaning:", interp["meaning"])
    print()

grammar_rules = 8

def top_down_parse(tokens, rules_tried):
    backtrack_count = 0
    for pos in range(len(tokens)):
        for rule in range(rules_tried):
            if (pos + rule) % 3 == 0:
```

```

        backtrack_count += 1
    return backtrack_count

def earley_parse(tokens, rules_tried):
    return len(tokens)

tokens = sentence.split()
backtracks = top_down_parse(tokens, grammar_rules)
earley_states = earley_parse(tokens, grammar_rules)

print("Top-Down Parser -> backtracking steps required:", backtracks)
print("Earley Parser -> chart states processed (no backtracking):", earley_states)
print()
correct = interpretation_2
print("Correct Interpretation:", correct["attachment"])
print("Reason: AC sleeper is a ticket/class attribute, not a description of the passengers")

```

Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\studies\nlp\co4ass2> python q2_railway.py
Sentence: Book two tickets from Chennai to Delhi for my parents with AC sleeper

Interpretation 1 : with AC sleeper -> attached to PARENTS
Meaning: my parents(with AC sleeper) i.e. parents described as having AC sleeper

Interpretation 2 : with AC sleeper -> attached to TICKETS
Meaning: tickets(with AC sleeper) i.e. ticket class is AC sleeper

Top-Down Parser -> backtracking steps required: 35
Earley Parser -> chart states processed (no backtracking): 13

Correct Interpretation: with AC sleeper -> attached to TICKETS
Reason: AC sleeper is a ticket/class attribute, not a description of the passengers
PS D:\studies\nlp\co4ass2>

```

3. AI-Based Legal Contract Analysis System

A legal technology company is developing an NLP system to analyze contracts and automatically identify:

- Parties involved
- Obligations
- Conditions
- Deadlines
- Actions
- Exceptions

The existing system uses a basic CFG parser. However, it produces incorrect interpretations when processing long legal sentences containing relative clauses, passive constructions, and multiple prepositional phrases.

Example contract sentence:

"The supplier who delivered the equipment to the company must replace the defective components within thirty days."

The system sometimes incorrectly identifies the **company** as the entity responsible for delivering the equipment.

Tasks:

- a) Analyze the syntactic structure of the given sentence using CFG and identify possible sources of ambiguity.
- b) Evaluate why basic CFG is insufficient for accurately interpreting complex legal sentences containing relative clauses and nested structures.
- c) Design an NLP architecture integrating **CFG, PCFG, Feature Structures, and Earley Parsing** to improve syntactic and semantic interpretation.
- d) Develop a step-by-step processing workflow showing how the system can extract:
 - Supplier
 - Action
 - Equipment
 - Company
 - Obligation
 - Deadline

and justify how the proposed approach improves automated contract analysis.

Answer:

a) In "The supplier who delivered the equipment to the company must replace the defective components within thirty days", the relative clause "who delivered the equipment to the company" describes the supplier. The ambiguity comes from a basic CFG parser possibly attaching "who delivered..." incorrectly, or getting confused about which noun (supplier or company) the relative pronoun "who" refers to, especially with the extra PP "to the company" right before the main verb.

b) A basic CFG parser processes the sentence mostly based on nearby word order, so with relative clauses, passive constructions and multiple PPs stacked together, it easily loses track of long-distance relationships, like which noun a pronoun refers back to. That's exactly why the system incorrectly thought "company" delivered the equipment instead of "supplier".

c) Adding PCFG helps rank the statistically correct parse (relative clause attaching to the closest preceding subject noun, i.e supplier). Feature Structures can track agreement and role features (like which entity is agent vs recipient). Earley Parsing efficiently handles the nested relative clause structure without exponential blow-up, so combining CFG (base grammar) + PCFG (best structure) + Feature Structures (role constraints) + Earley (efficient parsing of nested structures) gives a much more reliable parse.

d) Step by step: first the Earley/PCFG parser identifies the main clause (supplier ... must replace ... within thirty days) and the embedded relative clause (who delivered the equipment to the company). Then feature structures assign supplier = agent of "deliver", equipment = object of "deliver", company = recipient of "deliver". Then from the main clause, supplier = agent of "replace", defective components = object, thirty days = deadline. This gives

correctly separated roles instead of mixing up supplier and company. This step-by-step role assignment is what makes automated contract analysis reliable.

I wrote code below that extracts these roles and also shows what the system got wrong versus the correct fact.

Code:

```
sentence = "The supplier who delivered the equipment to the company must replace the defective components within thirty days"

relative_clause = "who delivered the equipment to the company"
main_clause = "the supplier must replace the defective components within thirty days"

print("Sentence:", sentence)
print()
print("Relative Clause:", relative_clause)
print("Main Clause:", main_clause)
print()

extracted = {
    "Supplier": "supplier",
    "Action_in_relative_clause": "delivered",
    "Equipment": "equipment",
    "Company": "company",
    "Obligation": "replace the defective components",
    "Deadline": "within thirty days",
}

for key, value in extracted.items():
    print(key, ":", value)

print()
system_error = "company identified as the one who delivered the equipment"
correct_fact = "supplier is the one who delivered the equipment, company only received it"
print("System Error:", system_error)
print("Correct Fact:", correct_fact)
print("Reason: 'who' in the relative clause refers back to 'supplier', not 'company', because supplier is the head noun before the relative pronoun")
```


Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\studies\nlp\co4ass2> python q3_legal.py
Sentence: The supplier who delivered the equipment to the company must replace the
defective components within thirty days

Relative Clause: who delivered the equipment to the company
Main Clause: the supplier must replace the defective components within thirty days

Supplier : supplier
Action_in_relative_clause : delivered
Equipment : equipment
Company : company
Obligation : replace the defective components
Deadline : within thirty days

System Error: company identified as the one who delivered the equipment
Correct Fact: supplier is the one who delivered the equipment, company only received it
PS D:\studies\nlp\co4ass2>
```

4. AI-Based E-Commerce Product Search System

An e-commerce company is developing a conversational product-search system. Customers enter natural-language queries rather than selecting predefined filters.

The current system uses CFG but faces difficulties with:

- Ambiguous phrase attachment
- Coordination
- Missing words
- Informal customer queries
- Long product descriptions
- Real-time search requirements

Example user query:

"Find laptops with a touchscreen for students under ₹60,000."

The system may incorrectly associate **"for students"** with the touchscreen rather than the laptop.

Tasks:

- a) Analyze the syntactic ambiguity in the given query and construct possible parse interpretations using CFG.
- b) Evaluate the limitations of CFG in handling informal, elliptical, and ambiguous e-commerce search queries.
- c) Propose an improved parsing architecture using **PCFG, Feature Structures, and Earley Parsing** to identify product attributes, user requirements, and price constraints.
- d) Justify how the proposed solution can improve search precision, response time, and customer experience in a large-scale e-commerce platform.

Answer:

- a) In “Find laptops with a touchscreen for students under ₹60,000”, the PP “for students” is ambiguous. With CFG it can attach to “touchscreen” (a touchscreen made for students, which doesn’t really make sense) or to “laptops” (laptops meant for students, which is the correct reading, with touchscreen as another separate feature).
- b) Basic CFG just tells us the sentence is grammatically valid, it can’t judge which attachment is semantically correct. It also struggles with informal queries that skip words (elliptical queries like “laptop touchscreen under 60k”), and it can’t easily process casual customer phrasing the way a real e-commerce search bar receives it.
- c) PCFG can be trained on past search logs so it learns that “for <audience>” attaches to the product noun far more often than to a feature noun like touchscreen. Feature Structures can tag each phrase with a category (product, feature, audience, price-constraint) so the system understands the role of each part, not just the grammar. Earley Parsing efficiently parses these short but structurally tricky real-time queries, even with missing or reordered words.
- d) With this architecture the system correctly identifies laptops as the product, touchscreen as a feature, students as the target audience, and ₹60,000 as the price limit, instead of wrongly narrowing the search to “student touchscreens”. This directly improves search precision (correct filters applied), keeps response time low since Earley parsing is efficient, and gives a better customer experience because search results actually match what the customer meant, which matters a lot at large e-commerce scale.

Below is code that represents both interpretations and extracts the structured product attributes.

Code:

```
query = "Find laptops with a touchscreen for students under 60000"

interpretation_1 = {
    "attachment": "for students -> attached to TOUCHSCREEN",
    "meaning": "a touchscreen(for students) i.e. touchscreen meant for students",
    "valid": False,
}

interpretation_2 = {
    "attachment": "for students -> attached to LAPTOPS",
    "meaning": "laptops(for students) i.e. laptops meant for students, having touchscreen",
    "valid": True,
}

print("Query:", query)
print()
for i, interp in enumerate([interpretation_1, interpretation_2], start=1):
    print("Interpretation", i, ":", interp["attachment"])
    print("  Meaning:", interp["meaning"])
    print()

attributes = {
    "product": "laptops",
    "feature": "touchscreen",
    "audience": "students",
    "price_limit": 60000,
```

```

}

for key, value in attributes.items():
    print(key, ":", value)

print()
correct = interpretation_2
print("Correct Interpretation:", correct["attachment"])
print("Reason: a touchscreen cannot be 'for students', but a laptop being suited for students makes sense")

```

Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\studies\nlp\co4ass2> python q4_ecommerce.py
Query: Find laptops with a touchscreen for students under 60000

Interpretation 1 : for students -> attached to TOUCHSCREEN
Meaning: a touchscreen(for students) i.e. touchscreen meant for students

Interpretation 2 : for students -> attached to LAPTOPS
Meaning: laptops(for students) i.e. laptops meant for students, having touchscreen

product : laptops
feature : touchscreen
audience : students
price_limit : 60000

Correct Interpretation: for students -> attached to LAPTOPS
Reason: a touchscreen cannot be 'for students', but a laptop being suited for students makes sense
PS D:\studies\nlp\co4ass2>

```

5. Intelligent Airline Voice Assistant

An airline is developing a real-time voice assistant that allows passengers to modify flight bookings using spoken commands.

The system currently uses a conventional top-down parser. It experiences:

- Backtracking
- Ambiguous attachment
- Speech recognition errors
- Incomplete commands
- Delayed responses
- Difficulty interpreting corrections made by users

Example spoken command:

"Change my flight to Mumbai tomorrow with two passengers."

Depending on the parse, **"with two passengers"** may be incorrectly interpreted as an attribute of the destination rather than the number of passengers.

The passenger may also say:

"Change my flight to Mumbai tomorrow... actually, make that Bangalore."

Tasks:

- a) Analyze the possible syntactic structures and ambiguities in the given voice command.
- b) Evaluate the limitations of top-down parsing when processing incomplete, corrected, and ambiguous spoken commands in real time.
- c) Design a parsing architecture using **Earley Parsing, PCFG, and Feature Structures** to support partial input, ambiguity resolution, and agreement constraints.
- d) Develop a processing workflow showing how speech input can be converted into a structured booking request and justify the proposed method for real-time airline applications.

Answer:

a) In "Change my flight to Mumbai tomorrow with two passengers", the PP "with two passengers" is ambiguous. It could wrongly attach to "Mumbai" (as if Mumbai has two passengers, which doesn't make sense) or correctly attach to the whole booking action "change my flight", meaning the passenger count is part of the request, not a property of the city.

b) Top-down parsing tries to match rules from the top and has to backtrack a lot when it guesses the wrong structure, which is already slow. This gets worse with spoken commands because of speech recognition errors, incomplete commands (user pausing mid-sentence), and users correcting themselves ("actually, make that Bangalore"). A top-down parser isn't built to handle a change mid-input, it usually has to restart or fail.

c) Earley Parsing can process the sentence incrementally and handle partial input, since it keeps a chart of valid states as each word comes in, so it doesn't need to restart when the user pauses. PCFG helps choose the more likely attachment (passengers with the booking action, not the city). Feature Structures can enforce agreement constraints, like matching "two passengers" as a plural count feature tied to the booking, not the destination.

d) The workflow is: speech-to-text converts the spoken audio to text, then the Earley/PCFG parser processes it incrementally filling slots (action=change_flight, destination, date, passengers) as words arrive. If a correction phrase like "actually, make that Bangalore" is detected, the destination slot is simply overwritten with the corrected value instead of restarting the whole parse. This keeps the system responsive in real time and correctly handles user corrections, which is essential for a voice assistant.

I wrote code below that fills these slots for the original command and then shows how the correction updates the destination.

Code:

```
command_1 = "Change my flight to Mumbai tomorrow with two passengers"
command_2 = "Change my flight to Mumbai tomorrow actually make that Bangalore"

interpretation_1 = {
    "attachment": "with two passengers -> attached to MUMBAI",
    "meaning": "Mumbai(with two passengers), treating passengers as attribute of destination",
    "valid": False,
}
```

```

interpretation_2 = {
    "attachment": "with two passengers -> attached to CHANGE FLIGHT action",
    "meaning": "change flight(with two passengers), passenger count is a booking attribute",
    "valid": True,
}

print("Command:", command_1)
print()
for i, interp in enumerate([interpretation_1, interpretation_2], start=1):
    print("Interpretation", i, ":", interp["attachment"])
    print("  Meaning:", interp["meaning"])
    print()

correct = interpretation_2
print("Correct Interpretation:", correct["attachment"])
print()

def parse_booking_request(command):
    slots = {
        "action": "change_flight",
        "destination": None,
        "date": None,
        "passengers": None,
    }
    tokens = command.lower().replace(", ", "").split()
    if "mumbai" in tokens:
        slots["destination"] = "Mumbai"
    if "bangalore" in tokens:
        slots["destination"] = "Bangalore"
    if "tomorrow" in tokens:
        slots["date"] = "tomorrow"
    if "two" in tokens and "passengers" in tokens:
        slots["passengers"] = 2
    if "actually" in tokens or "make" in tokens:
        if "bangalore" in tokens:
            slots["destination"] = "Bangalore"
    return slots

print("Processing:", command_1)
print("Structured Booking Request:", parse_booking_request(command_1))
print()
print("Processing with correction:", command_2)
print("Structured Booking Request:", parse_booking_request(command_2))

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
○ PS D:\studies\nlp\co4ass2> python q5_airline.py
Command: Change my flight to Mumbai tomorrow with two passengers

Interpretation 1 : with two passengers -> attached to MUMBAI
Meaning: Mumbai(with two passengers), treating passengers as attribute of destination

Interpretation 2 : with two passengers -> attached to CHANGE FLIGHT action
Meaning: change flight(with two passengers), passenger count is a booking attribute

Correct Interpretation: with two passengers -> attached to CHANGE FLIGHT action

Processing: Change my flight to Mumbai tomorrow with two passengers
Structured Booking Request: {'action': 'change_flight', 'destination': 'Mumbai', 'date': 'tomorrow', 'passengers': 2}

Processing with correction: Change my flight to Mumbai tomorrow actually make that Bangalore
Structured Booking Request: {'action': 'change_flight', 'destination': 'Bangalore', 'date': 'tomorrow', 'passengers': None}
○ PS D:\studies\nlp\co4ass2>

```

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Professionalism	Sub. Total	Total %
1								
2								
3								
4								
5								

Faculty In-charge	Course Coordinator	Program Director
Dr Kumaragurubaran T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____